23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No :D11
Team Members :

Roll No	Student Name
AM.SC.U4CSE25338	NIHARIKA S H
AM.SC.U4CSE25330	MADHAV S THAMPY
AM.SC.U4CSE25323	ISHAAN S VANNERI

Project Title : E-Commerce Platform

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	3
2	Introduction	3
3	Requirements (Functional & Non-functional)	5
4	UI Design	6
5	Database Design	12
6	Database Selection & Justification	22
7	Implementation	23
8	Testing & Validation	28
9	Results & Screenshots	29
10	Challenges faced & solutions	34
11	Conclusion & future enhancements	35
	References	35

1. Abstract

HELIOS is a web-based online watch marketplace developed to simplify the process of browsing and purchasing watches from different brands and categories. Traditional shopping methods and unorganized product information can make it difficult for customers to compare products and manage their purchases efficiently. The main objective of this project is to develop a user-friendly online platform that provides organized product information and supports the complete shopping process. The proposed system is implemented using HTML, CSS, and JavaScript for the frontend, Python Flask for the backend, and PostgreSQL for database management. The database consists of 12 related tables that manage users, brands, categories, products, carts, favourites, addresses, orders, order items, and payments through primary keys, foreign keys, constraints, and normalized relations. The system supports user authentication, product search and filtering, cart management, favourites, checkout, and order history. The completed application is deployed using Render, with its PostgreSQL database hosted on Neon. The project demonstrates the practical use of database management concepts to build a functional, reliable, and organized online watch marketplace.

2. Introduction

2.1 Problem Domain and Background

E-commerce has become an important part of modern shopping, allowing customers to explore products, compare prices, and make purchases through online platforms. Watches are available in different brands, styles, categories, and price ranges. Customers often need to compare product specifications, prices, materials, and features before choosing a suitable watch. Managing this information efficiently is important for providing a smooth shopping experience.

The HELIOS Online Watch Marketplace is a database-driven web application designed to organize watch products and support online shopping activities. It provides a centralized platform where customers can browse watches from different brands, view product details, add items to their shopping carts, save favourites, and place orders. The project focuses on applying database management concepts to a practical e-commerce application.

2.2 Existing System and Its Limitations

In a traditional shopping environment, customers may need to visit multiple stores to compare watches from different brands. Information about product availability, prices, and specifications may be spread across different sources, making comparison time-consuming.

Although existing e-commerce platforms provide online shopping facilities, managing product information, customer records, inventory, shopping carts, orders, and payments requires a properly designed database system. Without an organized database, applications may experience data duplication, inconsistent records, difficulties in retrieving information, and errors when maintaining relationships between customers, products, and orders.

The major limitations addressed by this project include:

- Difficulty in accessing watch information from different brands in one place.
- Time-consuming product search and comparison.
- The need for organized management of customer and product information.
- The possibility of inconsistent or duplicated data when information is not properly structured.
- The need to maintain shopping cart items, favourites, orders, and payment records systematically.
- The need to track product stock and maintain order information accurately.

2.3 Need for the Proposed System

The HELIOS system is developed to provide a centralized and organized online watch shopping experience. It aims to simplify product discovery, improve the management of shopping activities, and maintain data consistency through a relational database.

The system uses PostgreSQL to store related information in separate tables and establish relationships using primary keys and foreign keys. Database constraints help enforce data integrity, while normalization helps reduce unnecessary data redundancy. The Flask backend connects the website interface to the database and processes operations such as user authentication, product retrieval, cart management, checkout, and order placement.

2.4 Proposed System

HELIOS is implemented using HTML, CSS, and JavaScript for the user interface, Python Flask for backend processing, and PostgreSQL for persistent data storage. The application is deployed on Render, while its PostgreSQL database is hosted on Neon.

The system provides the following features:

- **User authentication:** Allows users to register and log in.
- **Product browsing and search:** Displays watches and supports product search and category filtering.
- **Product details:** Provides information such as price, brand, stock, and watch specifications.
- **Shopping cart and favourites:** Allows users to manage selected products and save preferred watches.
- **Checkout and order placement:** Supports address selection or entry, payment method selection, and order creation.
- **Order history:** Allows users to view their previous orders and related details.
- **Relational database management:** Organizes user, product, cart, address, order, and payment information through related tables and constraints.

2.5 How the Proposed System Addresses the Problem

The proposed system brings watch information and shopping operations together in one web application. Customers can search for watches, view their specifications, manage their carts and favourites, and place orders without needing to visit multiple physical stores for product comparison.

The relational database organizes information into 12 related tables, reducing duplication and maintaining consistency between records. Primary keys uniquely identify records, while foreign keys establish relationships between entities. Constraints help prevent invalid data from being stored. The application also connects the frontend, backend, and database so that users can interact with stored information through a common interface.

Thus, HELIOS demonstrates how database management concepts can be applied to develop an organized online marketplace that supports product browsing, shopping operations, and order management.

3. Requirements (Functional and Non-functional)

3.1 Functional Requirements

Functional requirements describe the operations that the HELIOS Online Watch Marketplace must perform.

- **User Registration and Authentication:** The system allows users to create accounts and log in using their credentials. User information is stored in the database.
- **Product Management and Browsing:** The system retrieves and displays watch details, including brand, price, stock, and specifications. Products can be searched and filtered by category.
- **Shopping Cart Management:** Users can add products to their carts and manage selected items before checkout.
- **Favourites Management:** Users can save watches to their favourites and access them later.
- **Checkout and Order Placement:** The system collects the required delivery details, supports UPI, card, and cash-on-delivery payment methods, and stores order information in the database.
- **Order History:** Users can view their previously placed orders and associated details.
- **Database Operations:** The application performs database operations such as inserting, retrieving, and updating records as required by registration, shopping, and order processing. Database constraints maintain data integrity and consistency.

3.2 Non-functional Requirements

Non-functional requirements describe the quality and operational characteristics of the system.

- **Performance:** Product information and user requests should be processed within a reasonable time.
- **Security:** User authentication and password hashing help protect account information. Database credentials are configured through environment variables in the deployed application.
- **Reliability:** The system should maintain consistent records and handle database operations correctly.
- **Usability:** The interface should be simple, understandable, and easy to navigate for customers.
- **Data Integrity:** Primary keys, foreign keys, unique constraints, and other applicable constraints help prevent invalid or inconsistent data.
- **Scalability:** The application uses a separate web-hosting service and PostgreSQL database, providing a foundation for future improvements and expansion.
- **Maintainability:** The application separates frontend files, backend logic, and database schema to make future modifications easier.
- **Availability:** The deployed application should be accessible online when its hosting service is operational.

4. UI Design

4.1 Overview of the User Interface

The user interface of HELIOS – Online Watch Marketplace is designed to provide a simple and convenient online shopping experience. The application uses HTML, CSS, and JavaScript to create an interactive and responsive interface. The interface allows users to browse watches, search for products, view product specifications, manage their shopping carts and favourites, complete checkout, and view their orders.

The design emphasizes ease of navigation, clear product presentation, and convenient access to the main shopping functions. Product images, names, prices, and relevant details help customers explore the available watches and make informed purchasing decisions.

4.2 Navigation Flow

The navigation flow describes how users move between different screens while using the application.

The general navigation flow is as follows:

- **Home Page:** Users enter the website and view the main page and featured watch products.

- **Product Catalogue:** Users browse the available watches and use search or category filters to find suitable products.
- **Product Details:** Users select a watch to view its price, availability, brand, and specifications.
- **Login or Registration:** Users can create an account or log in to access account-related shopping functions.
- **Cart and Favourites:** Users add watches to their shopping cart or save them to their favourites list.
- **Checkout:** Users provide delivery details and select a payment method.
- **Order History:** After placing an order, users can access the order-history page to view their orders.

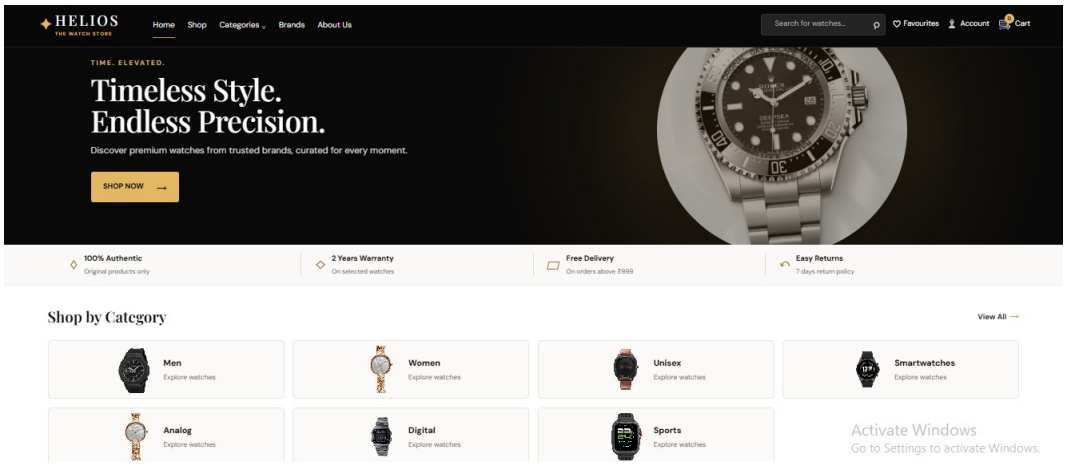
This navigation structure helps users complete the shopping process through a sequence of connected screens.

4.3 Home Page

The home page serves as the main entry point of the application. It introduces the HELIOS marketplace and provides access to product browsing and other major sections of the website.

The page includes navigation links, product search, category navigation, and watch listings. Users can select a product to open its details page or use the available navigation options to move to other sections.

The home page is designed to make product discovery convenient and to provide a clear starting point for customers.

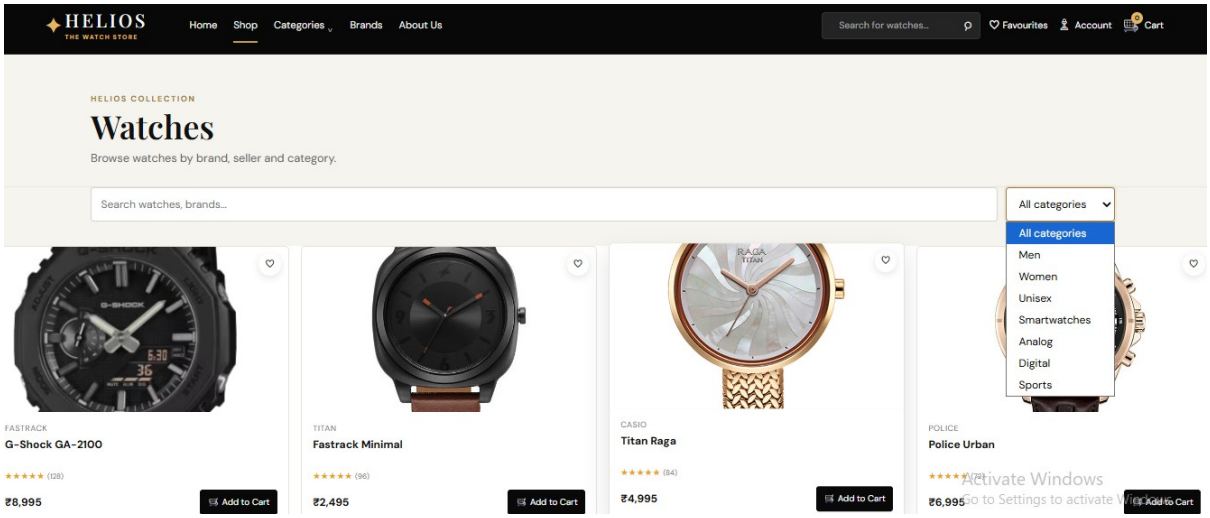


4.4 Product Catalogue and Search Screen

The product catalogue displays the watches available in the database. Each product listing presents relevant information such as the watch name, brand, price, and image.

The search functionality allows users to locate watches using search terms. Category filtering helps users narrow down the available products based on their selected category. These features reduce the effort required to find a particular watch.

Users can select a product from the catalogue to view its detailed information.

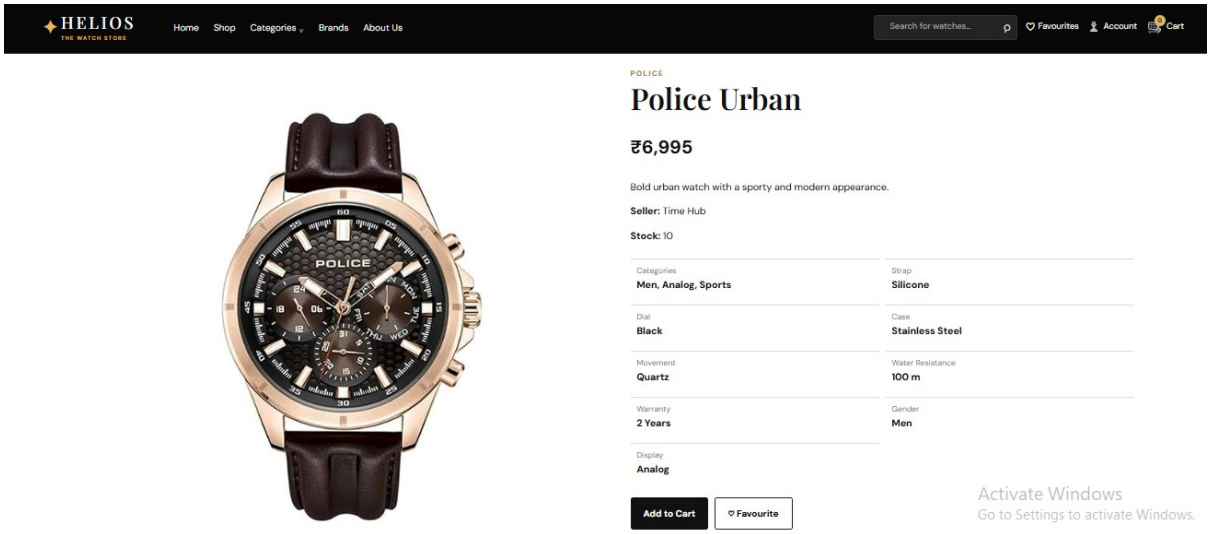


4.5 Product Details Screen

The product details screen provides more information about an individual watch. It displays product information such as the brand, price, stock availability, and specifications, including strap material, dial colour, case material, movement type, water resistance, warranty, gender, and display type, where available.

The screen allows users to understand the product before deciding whether to add it to their cart or save it to their favourites.

The information displayed on this screen is retrieved from the PostgreSQL database through the Flask backend.

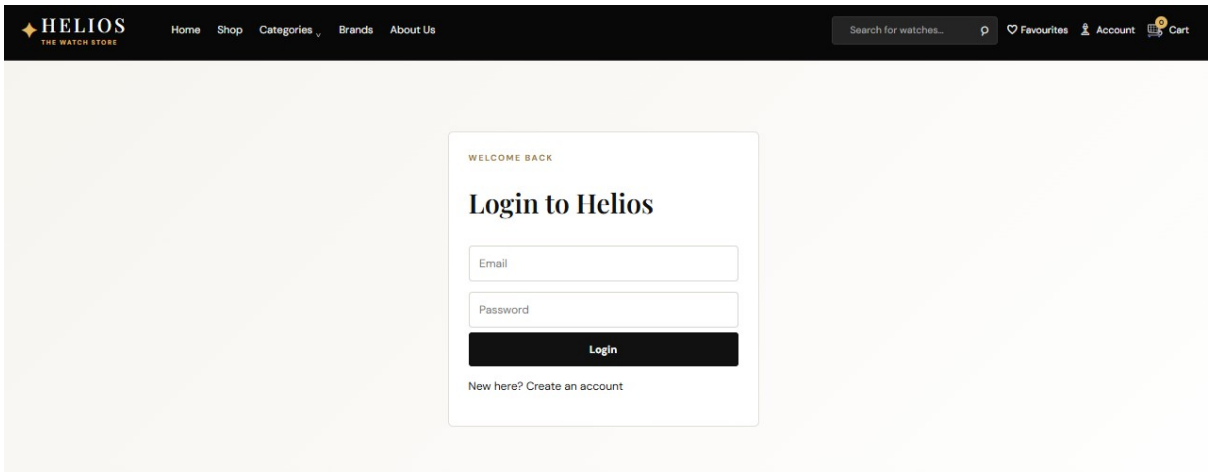


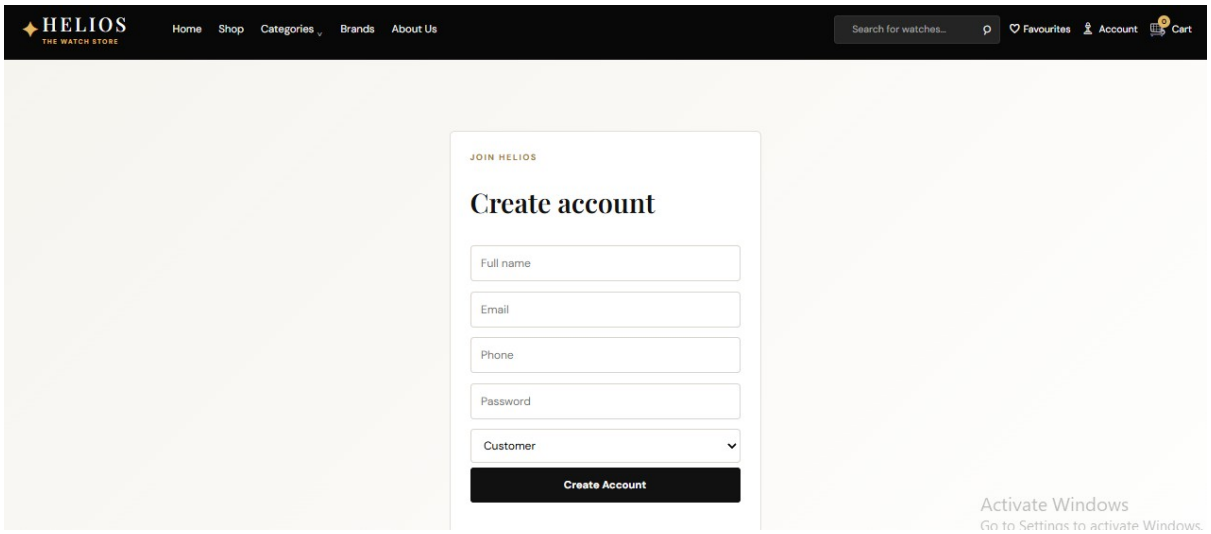
4.6 Login and Registration Screens

The login screen allows registered users to access their accounts by entering their credentials. The registration screen enables new users to create an account by providing the required information.

The backend validates the submitted credentials and interacts with the database to authenticate users or store new account information. Passwords are stored using password hashing rather than as plain text.

These screens provide the entry point for account-related functions such as managing favourites, using the shopping cart, and accessing order information.



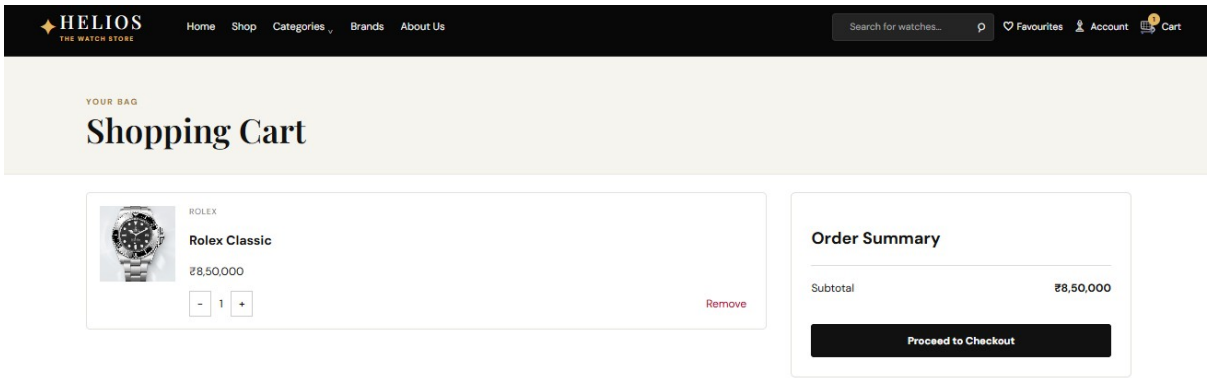


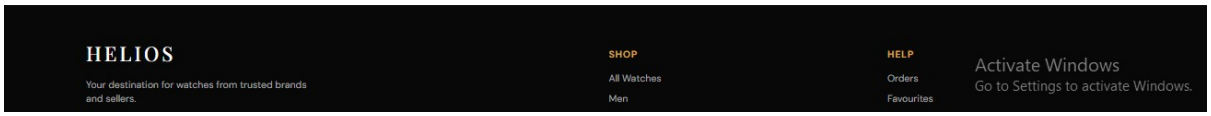
4.7 Shopping Cart and Favourites Screens

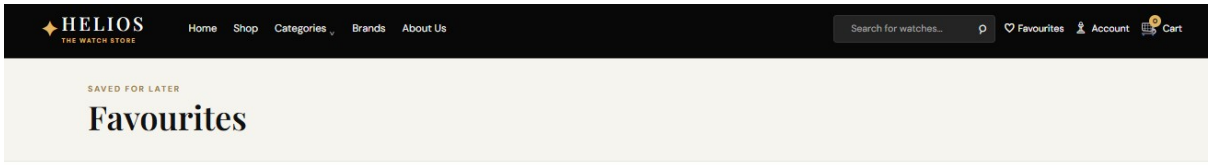
The shopping cart screen displays the products selected by the user for purchase. It allows users to review their selected items before proceeding to checkout. Cart information is maintained through the application's database operations.

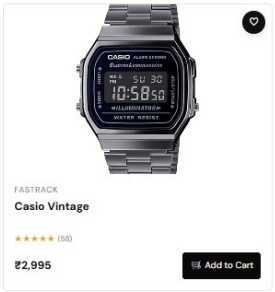
The favourites screen displays watches that users have saved for future reference. This feature makes it easier for users to revisit products without searching for them again.

Both screens help users organize their shopping activities before placing an order.







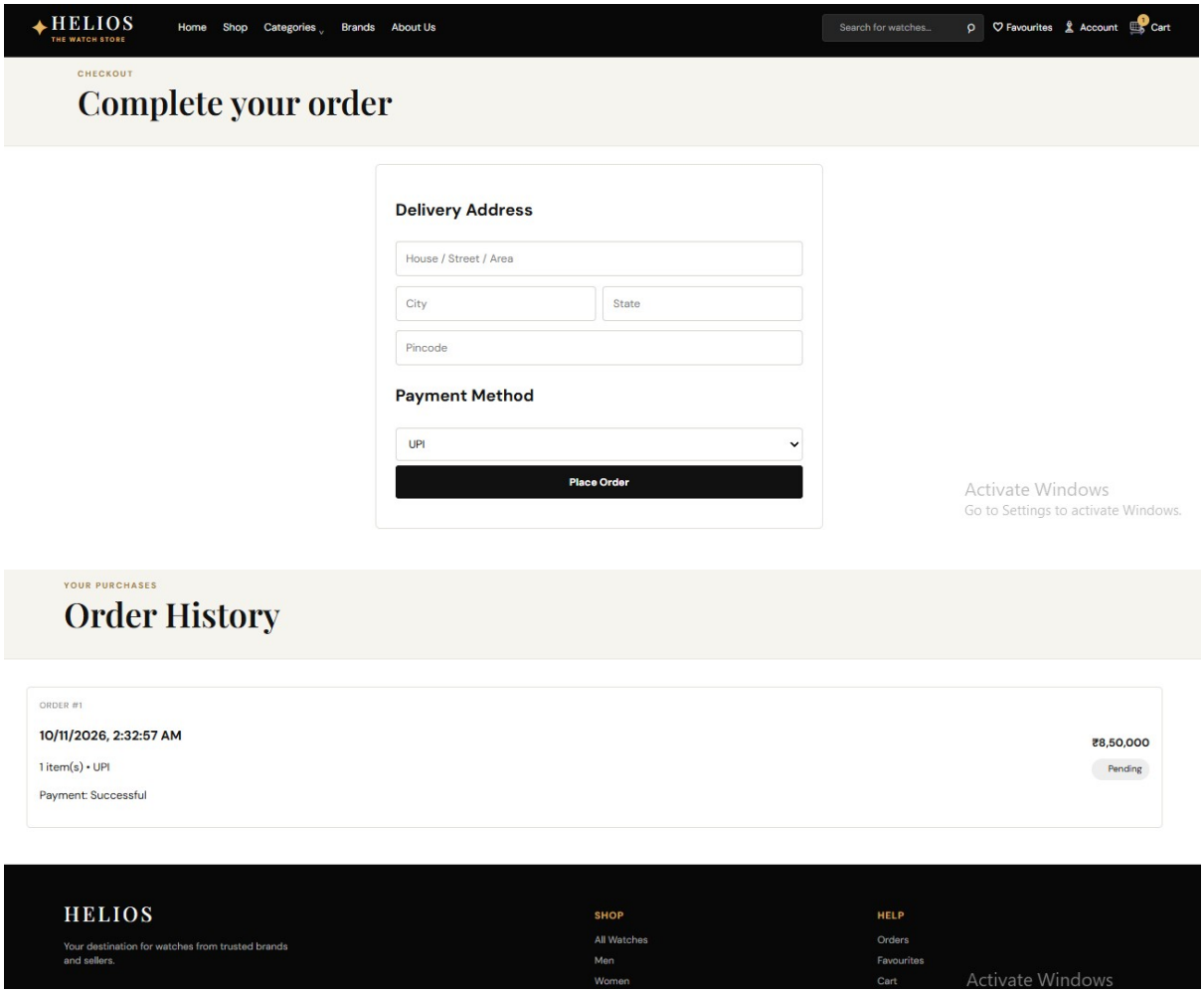


4.8 Checkout and Order History Screens

The checkout screen collects the information required to process an order, including delivery address details and the selected payment method. The application supports UPI, card, and cash on delivery as payment options.

After the required information is submitted, the backend processes the order and stores the relevant details in the database. The order-history screen allows users to view their previously placed orders and related information.

These screens connect the user interface with the database operations involved in order processing and order retrieval.



4.9 User Interaction and Interface Design

Users interact with HELIOS through navigation links, search fields, category filters, buttons, and forms. When a user performs an operation, JavaScript handles relevant interface interactions and communicates with the Flask backend where required. The backend retrieves or modifies information in PostgreSQL and returns the appropriate response to the interface.

The interface is organized into separate pages and supporting files to make navigation and maintenance easier. The website also uses responsive styling to improve usability across different screen sizes.

Overall, the UI design of HELIOS provides an organized interface for product discovery, account access, shopping cart management, checkout, and order history while connecting these functions to the underlying relational database.

5. Database Design

5.1 Overview

The HELIOS Online Watch Marketplace uses a relational database to store and manage information related to users, products, brands, categories, shopping carts, favourites, addresses, orders, and payments. The database is designed to maintain data consistency, reduce redundancy, and establish clear relationships between different entities.

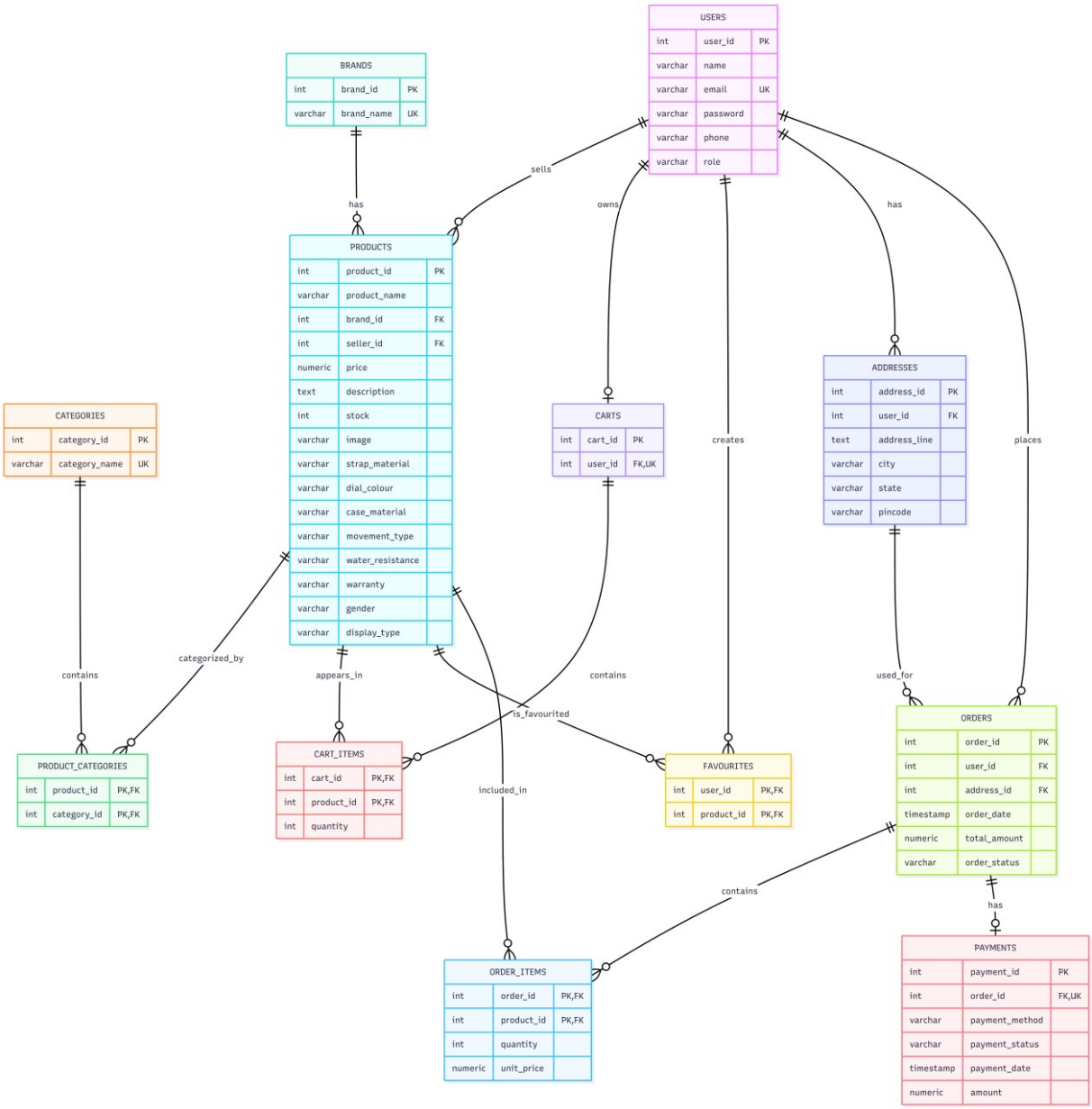
The database contains the following 12 relations:

- USERS
- BRANDS
- CATEGORIES
- PRODUCTS
- PRODUCT_CATEGORIES
- CARTS
- CART_ITEMS
- FAVOURITES
- ADDRESSES
- ORDERS
- ORDER_ITEMS
- PAYMENTS

The database uses primary keys to uniquely identify records and foreign keys to establish relationships between related tables. Appropriate constraints are used to maintain data integrity.

5.2 ER Model

The Entity-Relationship model represents the main entities of the HELIOS system and the relationships between them.



The major entities represented in the ER model are USERS, BRANDS, CATEGORIES, PRODUCTS, CARTS, CART_ITEMS, FAVOURITES, ADDRESSES, ORDERS, ORDER_ITEMS, and PAYMENTS. PRODUCT_CATEGORIES is used to represent the many-to-many relationship between products and categories.

5.3 Entities and Attributes

USERS

- **user_id** – Primary Key
- name
- email
- password
- phone
- role

The USERS relation stores customer, seller, and administrator information.

BRANDS

- **brand_id** – Primary Key
- brand_name

The BRANDS relation stores the brands associated with watches.

CATEGORIES

- **category_id** – Primary Key
- category_name

The CATEGORIES relation stores product categories such as Men, Women, Unisex, Smartwatches, Analog, Digital, and Sports.

PRODUCTS

- **product_id** – Primary Key
- product_name
- brand_id – Foreign Key
- seller_id – Foreign Key
- price
- description
- stock
- image
- strap_material
- dial_colour
- case_material
- movement_type
- water_resistance
- warranty
- gender
- display_type

The PRODUCTS relation stores the main information and specifications of each watch.

PRODUCT_CATEGORIES

- **product_id** – Primary Key, Foreign Key
- **category_id** – Primary Key, Foreign Key

This relation represents the many-to-many relationship between PRODUCTS and CATEGORIES.

CARTS

- **cart_id** – Primary Key
- user_id – Foreign Key

The CARTS relation stores the shopping cart associated with each customer.

CART_ITEMS

- **cart_id** – Primary Key, Foreign Key
- **product_id** – Primary Key, Foreign Key
- quantity

This relation stores the products and quantities contained in each cart.

FAVOURITES

- **user_id** – Primary Key, Foreign Key
- **product_id** – Primary Key, Foreign Key

This relation stores products saved as favourites by users.

ADDRESSES

- **address_id** – Primary Key
- user_id – Foreign Key
- address_line
- city
- state
- pincode

The ADDRESSES relation stores delivery addresses associated with users.

ORDERS

- **order_id** – Primary Key
- user_id – Foreign Key
- address_id – Foreign Key
- order_date
- total_amount
- order_status

The ORDERS relation stores information about customer orders.

ORDER_ITEMS

- **order_id** – Primary Key, Foreign Key
- **product_id** – Primary Key, Foreign Key
- quantity
- unit_price

This relation stores the products included in each order along with their quantity and purchase price.

PAYMENTS

- **payment_id** – Primary Key
- order_id – Foreign Key
- payment_method
- payment_status
- payment_date
- amount

The PAYMENTS relation stores payment information associated with orders.

5.4 Relationships and Cardinalities

The major relationships in the HELIOS database are:

Relationship	Cardinality
USERS → PRODUCTS through seller	1 : M
BRANDS → PRODUCTS	1 : M
PRODUCTS ↔ CATEGORIES	M : N
USERS → CARTS	1 : 0..1
CARTS ↔ PRODUCTS through CART_ITEMS	M : N
USERS ↔ PRODUCTS through FAVOURITES	M : N
USERS → ADDRESSES	1 : M
USERS → ORDERS	1 : M
ADDRESSES → ORDERS	1 : M
ORDERS ↔ PRODUCTS through ORDER_ITEMS	M : N
ORDERS → PAYMENTS	1 : 0..1

A seller can list multiple products, while each product is associated with one seller. Similarly, one brand can contain multiple products, while each product belongs to one brand.

Products and categories have a many-to-many relationship because one product can belong to multiple categories and one category can contain multiple products. This relationship is implemented using the PRODUCT_CATEGORIES relation.

The relationship between orders and products is also many-to-many and is implemented using ORDER_ITEMS, which stores quantity and unit price for each product in an order.

5.5 Primary Keys, Foreign Keys and Constraints

Primary keys uniquely identify each record in a relation. Composite primary keys are used where a combination of attributes is required to uniquely identify a record.

Foreign keys maintain referential integrity between related relations.

Important constraints include

- ✧ user_id is the primary key of USERS.
- ✧ email in USERS is UNIQUE and NOT NULL.
- ✧ brand_id is the primary key of BRANDS and brand_name is UNIQUE.
- ✧ category_id is the primary key of CATEGORIES and category_name is UNIQUE.
- ✧ product_id is the primary key of PRODUCTS.
- ✧ brand_id and seller_id in PRODUCTS are foreign keys.
- ✧ PRODUCT_CATEGORIES uses (product_id, category_id) as a composite primary key.
- ✧ CART_ITEMS uses (cart_id, product_id) as a composite primary key.
- ✧ FAVOURITES uses (user_id, product_id) as a composite primary key.
- ✧ ORDER_ITEMS uses (order_id, product_id) as a composite primary key.
- ✧ user_id in CARTS is UNIQUE so that a customer can have at most one cart.
- ✧ order_id in PAYMENTS is UNIQUE so that an order can have at most one payment record.
- ✧ Quantity and monetary values use appropriate CHECK constraints to prevent invalid values.
- ✧ Payment methods and payment statuses are restricted to valid values.
- ✧ Foreign keys prevent records from referring to non-existent users, products, orders, or other related entities.

5.6 Relational Schema

The ER model is converted into the following relational schema:

```
USERS(
    user_id PK,
    name,
    email UNIQUE,
    password,
    phone,
    role
)

BRANDS(
    brand_id PK,
    brand_name UNIQUE
```

```
)

CATEGORIES(
    category_id PK,
    category_name UNIQUE
)

PRODUCTS(
    product_id PK,
    product_name,
    brand_id FK,
    seller_id FK,
    price,
    description,
    stock,
    image,
    strap_material,
    dial_colour,
    case_material,
    movement_type,
    water_resistance,
    warranty,
    gender,
    display_type
)

PRODUCT_CATEGORIES(
    product_id PK, FK,
    category_id PK, FK
)

CARTS(
    cart_id PK,
    user_id FK UNIQUE
)

CART_ITEMS(
    cart_id PK, FK,
    product_id PK, FK,
    quantity
)

FAVOURITES(
    user_id PK, FK,
    product_id PK, FK
)

ADDRESSES(
    address_id PK,
    user_id FK,
    address_line,
    city,
    state,
    pincode
)

ORDERS(
    order_id PK,
    user_id FK,
    address_id FK,
    order_date,
```

```
    total_amount,
    order_status
)

ORDER_ITEMS(
    order_id PK, FK,
    product_id PK, FK,
    quantity,
    unit_price
)

PAYMENTS(
    payment_id PK,
    order_id FK UNIQUE,
    payment_method,
    payment_status,
    payment_date,
    amount
)
```

5.7 Conversion of ER Model into Relational Tables

The ER model was converted into relational tables using standard relational mapping rules.

Each strong entity in the ER model was converted into a separate relation. For example, USERS, BRANDS, CATEGORIES, PRODUCTS, ADDRESSES, ORDERS, and PAYMENTS were directly converted into their respective tables.

For one-to-many relationships, the primary key of the entity on the "one" side was included as a foreign key in the relation on the "many" side. For example, `brand_id` and `seller_id` are included in PRODUCTS.

Many-to-many relationships were converted into separate relations containing the primary keys of both participating entities. Therefore, PRODUCT_CATEGORIES represents the relationship between PRODUCTS and CATEGORIES, while CART_ITEMS, FAVOURITES, and ORDER_ITEMS represent their respective many-to-many relationships.

A unique foreign key is used in CARTS to represent the one-to-zero-or-one relationship between USERS and CARTS. Similarly, the unique `order_id` in PAYMENTS represents the one-to-zero-or-one relationship between ORDERS and PAYMENTS.

Thus, the ER model is transformed into a set of related relational tables while preserving the relationships and constraints of the original design.

5.8 Functional Dependencies

Functional dependencies describe how one attribute or a set of attributes determines other attributes in a relation.

The major functional dependencies in the HELIOS database are:

```
user_id → name, email, password, phone, role

brand_id → brand_name

category_id → category_name

product_id → product_name, brand_id, seller_id, price,
              description, stock, image, strap_material,
              dial_colour, case_material, movement_type,
              water_resistance, warranty, gender, display_type

cart_id → user_id

(cart_id, product_id) → quantity

(user_id, product_id) → no non-key attributes

address_id → user_id, address_line, city, state, pincode

order_id → user_id, address_id, order_date,
            total_amount, order_status

(order_id, product_id) → quantity, unit_price

payment_id → order_id, payment_method, payment_status,
             payment_date, amount
```

For example, `product_id` uniquely determines all the other product attributes. Similarly, `(order_id, product_id)` determines the quantity and unit price of a product within a particular order.

5.9 Candidate Keys

The candidate keys of the main relations are:

Relation	Candidate Key
USERS	user_id
BRANDS	brand_id
CATEGORIES	category_id
PRODUCTS	product_id
PRODUCT_CATEGORIES	(product_id, category_id)
CARTS	cart_id
CART_ITEMS	(cart_id, product_id)
FAVOURITES	(user_id, product_id)
ADDRESSES	address_id
ORDERS	order_id
ORDER_ITEMS	(order_id, product_id)
PAYMENTS	payment_id

Composite keys are required in relations such as CART_ITEMS and ORDER_ITEMS because a single attribute is not sufficient to uniquely identify each record.

5.10 Normalization

The HELIOS database is normalized up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and prevent insertion, update, and deletion anomalies.

First Normal Form (1NF)

The relations satisfy 1NF because:

All attributes contain atomic values.

There are no repeating groups.

There are no multi-valued attributes within a single field.

Each relation has a defined primary key.

Second Normal Form (2NF)

The relations satisfy 2NF because they are already in 1NF and there are no partial dependencies.

In relations with composite keys, non-key attributes depend on the complete composite key.

For example:

`(order_id, product_id) → quantity, unit_price`

Both `order_id` and `product_id` together are required to identify the corresponding quantity and unit price.

3NF

The relations satisfy 3NF because they are already in 2NF and there are no transitive dependencies between non-key attributes.

For example, brand information is stored in BRANDS instead of being repeatedly stored with every product. Similarly, category information is stored separately in CATEGORIES.

This reduces unnecessary duplication and makes updates more consistent.

BCNF

The HELIOS database satisfies BCNF because every non-trivial functional dependency has a determinant that is a candidate key.

For example:

`user_id → name, email, password, phone, role`

Here, `user_id` is a candidate key of USERS.

Similarly:

`(order_id, product_id) → quantity, unit_price`

Here, `(order_id, product_id)` is the candidate key of ORDER_ITEMS.

Therefore, the database design satisfies BCNF and minimizes redundancy and common data anomalies.

5.11 Summary

The HELIOS database design provides a structured relational model for managing the operations of the online watch marketplace. The use of primary keys, foreign keys, composite keys, unique constraints, and check constraints maintains data integrity. Many-to-many relationships are handled through separate associative relations, while normalization up to BCNF reduces redundancy and helps prevent insertion, update, and deletion anomalies.

6. Database Selection & Justification

6.1 Selected Database Management System

The DBMS selected for implementing the HELIOS Online Watch Marketplace is **PostgreSQL**. PostgreSQL is an open-source object-relational database management system that supports SQL and provides features required for developing reliable and scalable database-driven applications.

6.2 Justification for Selecting PostgreSQL

PostgreSQL was selected because it is well suited for managing the structured and related data used in the HELIOS application.

- ✧ **Performance:** PostgreSQL provides efficient data retrieval and processing for operations such as product search, filtering, cart management, and order retrieval.
- ✧ **Scalability:** PostgreSQL can handle increasing amounts of data and users, making it suitable for future expansion of the HELIOS marketplace.
- ✧ **Data Integrity:** Primary keys, foreign keys, unique constraints, check constraints, and transactions help maintain accurate and consistent database records.
- ✧ **Security:** PostgreSQL provides access control and secure database management features. In the HELIOS application, database credentials are stored using environment variables rather than being directly exposed in the application code.

- ✧ **SQL Support:** PostgreSQL provides strong support for standard SQL operations such as `SELECT`, `INSERT`, `UPDATE`, `DELETE`, joins, constraints, and aggregate operations.
- ✧ **Open Source and Cost:** PostgreSQL is open-source and can be used without licensing costs, making it suitable for an academic project.
- ✧ **Integration:** PostgreSQL integrates well with Python and Flask through the `psycopg` database adapter, allowing the web application to communicate efficiently with the database.
- ✧ **Reliability:** PostgreSQL provides transaction support and mechanisms for maintaining consistency during database operations such as order placement and payment processing.

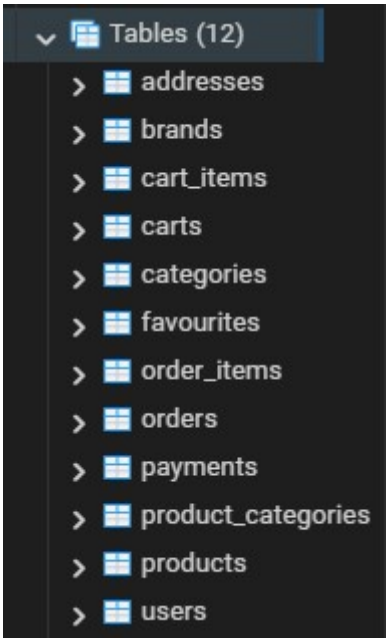
Therefore, PostgreSQL was chosen as the DBMS for HELIOS because it provides the required SQL functionality, data integrity, security, reliability, scalability, and easy integration with the Flask backend while remaining cost-effective.

.

7. Implementation

7.1 Database Creation

The HELIOS Online Watch Marketplace uses PostgreSQL as its relational database management system. The database was created and configured for storing and managing information related to users, products, product_categories, brands, categories, carts, cart_items, favourites, addresses, orders, order_items, and payments..

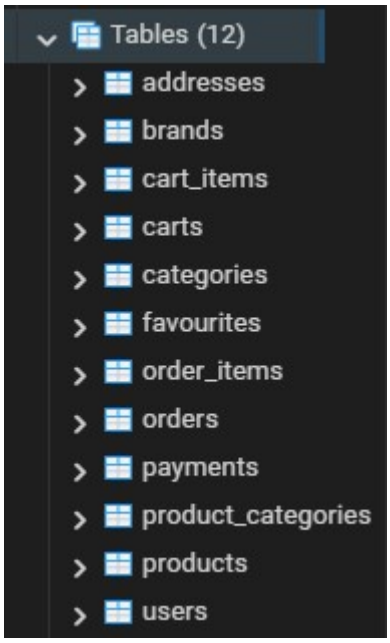


The tables are connected using primary keys and foreign keys to maintain relationships and data integrity.

7.2 Table Creation

The database schema was implemented using SQL `CREATE TABLE` statements. The HELIOS database contains 12 related tables: USERS, BRANDS, CATEGORIES, PRODUCTS, PRODUCT_CATEGORIES, CARTS, CART_ITEMS, FAVOURITES, ADDRESSES, ORDERS, ORDER_ITEMS, and PAYMENTS.

Primary keys, foreign keys, unique constraints, NOT NULL constraints, and CHECK constraints were defined while creating the tables to maintain data integrity.



The following SQL command is an example of table creation used in the HELIOS database.

```
CREATE TABLE brands (
    brand_id SERIAL PRIMARY KEY,
    brand_name VARCHAR(100) UNIQUE NOT NULL
);
```

This table stores the brands available in the marketplace.

Similarly, the PRODUCTS table was created with references to the BRANDS and USERS tables.

```
CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(150) NOT NULL,
    brand_id INT NOT NULL REFERENCES brands(brand_id),
    seller_id INT NOT NULL REFERENCES users(user_id),
    price NUMERIC(10,2) NOT NULL,
    stock INT NOT NULL,
    image VARCHAR(255)
);
```

7.3 Constraints

Constraints were used to maintain data accuracy and consistency.

The main constraints used in the database include:

PRIMARY KEY: Uniquely identifies each record.

FOREIGN KEY: Maintains relationships between tables.

UNIQUE: Prevents duplicate values such as email addresses and brand names.

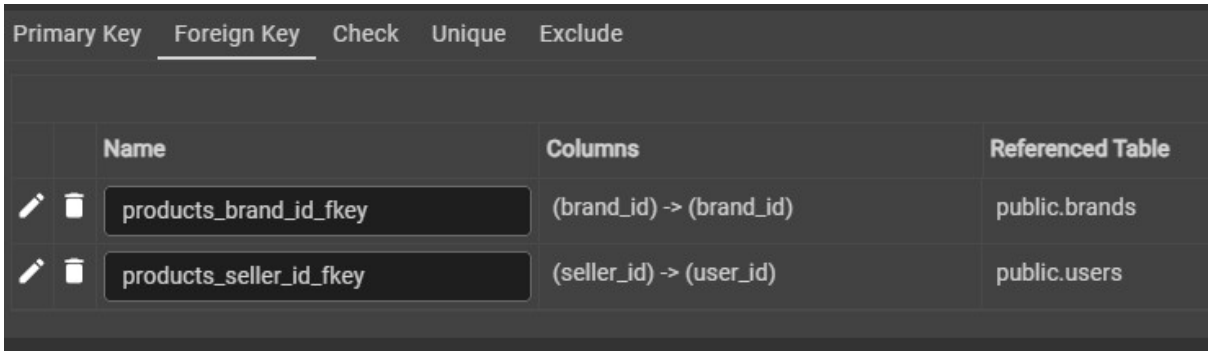
NOT NULL: Ensures that required fields contain values.

CHECK: Restricts values to valid ranges or predefined options.

For example, the payment table restricts the payment method to supported options.

```
CHECK (payment_method IN ('UPI', 'Card', 'Cash on Delivery'))
```

Similarly, order status is restricted to valid values such as Pending, Confirmed, Shipped, Delivered, and Cancelled.



brand_id INT NOT NULL REFERENCES brands(brand_id),

seller_id INT NOT NULL REFERENCES users(user_id)

Foreign key constraints are used to maintain referential integrity between the PRODUCTS table and the BRANDS and USERS tables.

7.4 Sample Data

Sample records were inserted into the database to test the functionality of the application.

For example:

```
INSERT INTO brands (brand_name)
VALUES ('Casio'), ('Fastrack'), ('Titan'), ('Police'), ('Fossil'),
('Rolex');
```

Sample products were also inserted with information such as product name, brand, seller, price, stock, and specifications.

7.5 CRUD Operations

The application performs CRUD operations through the Flask backend and PostgreSQL database.

Create

New users and orders can be inserted into the database.

```
INSERT INTO brands (brand_name)
VALUES ('Test Brand');
```

	brand_id [PK] integer	brand_name character varying (100)
1	7	Test Brand

Read

Product information can be retrieved using a SELECT query.

```
SELECT product_id, product_name, price, stock
FROM products;
```

	product_id [PK] integer	product_name character varying (150)	price numeric (10,2)	stock integer
1	3	Titan Raga	4995.00	12
2	4	Police Urban	6995.00	10
3	5	Fossil Gen 6	18995.00	8
4	6	Casio Vintage	2995.00	20
5	7	Rolex Classic	850000.00	3
6	8	Fastrack Active	3995.00	18
7	2	Fastrack Minimal	2495.00	24
8	1	G-Shock GA-2100	8995.00	14

Update

Product stock can be updated after a purchase.

```
UPDATE products
SET stock = stock - 1
WHERE product_id = 1;
```

Output:

```
UPDATE 1
```

Delete

Records can be deleted when required, subject to the defined relationships and constraints.

```
DELETE FROM favourites
WHERE user_id = 1 AND product_id = 1;
```

Output:

```
DELETE 1
```

7.6 Join Queries

Joins are used to retrieve related information from multiple tables.

For example, the following query retrieves product names along with their corresponding brand names:

```
SELECT p.product_name, b.brand_name, p.price
FROM products p
JOIN brands b
ON p.brand_id = b.brand_id;
```

Sample Output:

	product_name character varying (150)	brand_name character varying (100)	price numeric (10,2)
1	G-Shock GA-2100	CASIO	8995.00
2	Casio Vintage	CASIO	2995.00
3	Fastrack Minimal	FASTRACK	2495.00
4	Fastrack Active	FASTRACK	3995.00
5	Titan Raga	TITAN	4995.00
6	Police Urban	POLICE	6995.00
7	Fossil Gen 6	FOSSIL	18995.00
8	Rolex Classic	ROLEX	850000.00

Another join can be used to retrieve order details along with the products included in an order:

```
SELECT o.order_id, p.product_name,
       oi.quantity, oi.unit_price
FROM orders o
JOIN order_items oi
ON o.order_id = oi.order_id
JOIN products p
ON oi.product_id = p.product_id;
```

This query combines information from ORDERS, ORDER_ITEMS, and PRODUCTS to display the products associated with each order.

7.7 Application Integration

The PostgreSQL database is connected to the Flask backend using the `psycopg` Python library. The frontend is developed using HTML, CSS, and JavaScript, while Flask handles user requests and performs the required database operations.

The application retrieves and manages data for user authentication, product browsing, search and filtering, favourites, shopping cart management, checkout, order placement, and order history. The results retrieved from the database are returned to the frontend and displayed to the users.

The application is deployed on Render, while the PostgreSQL database is hosted on Neon. This integration enables the different components to work together to provide a functional, database-driven online watch marketplace.

8. Testing & Validation

Testing was performed to verify the functionality of the HELIOS Online Watch Marketplace. The major features of the application were tested through the user interface to ensure that users could browse products, manage their shopping activities, and place orders successfully.

8.1 Test Cases

Test Case	Expected Output	Actual Output	Result
Homepage	Homepage loads correctly	Homepage displayed successfully	Pass
Product Catalogue	Available watches are displayed	Product catalogue displayed correctly	Pass
Product Details	Selected watch information is displayed	Product details displayed correctly	Pass
Search and Filtering	Relevant watches are displayed	Search and filtering worked correctly	Pass
User Registration	A new user can register	Registration worked successfully	Pass
User Login	Registered user can log in	Login worked successfully	Pass
Favourites	Selected watches can be added to favourites	Favourites functionality worked correctly	Pass

Shopping Cart	Selected watches can be added to the cart	Cart functionality worked correctly	Pass
Checkout	Order details can be submitted	Checkout completed successfully	Pass
Order History	Placed orders can be viewed	Order history displayed correctly	Pass

8.2 Validation and Data Integrity

The PostgreSQL database uses primary keys to uniquely identify records and foreign keys to maintain relationships between related tables. Constraints such as NOT NULL, UNIQUE, and CHECK are used where defined in the database schema to maintain valid and consistent data.

The application also validates required checkout information, including the address, city, state, and PIN code, before submitting an order.

8.3 Testing Summary

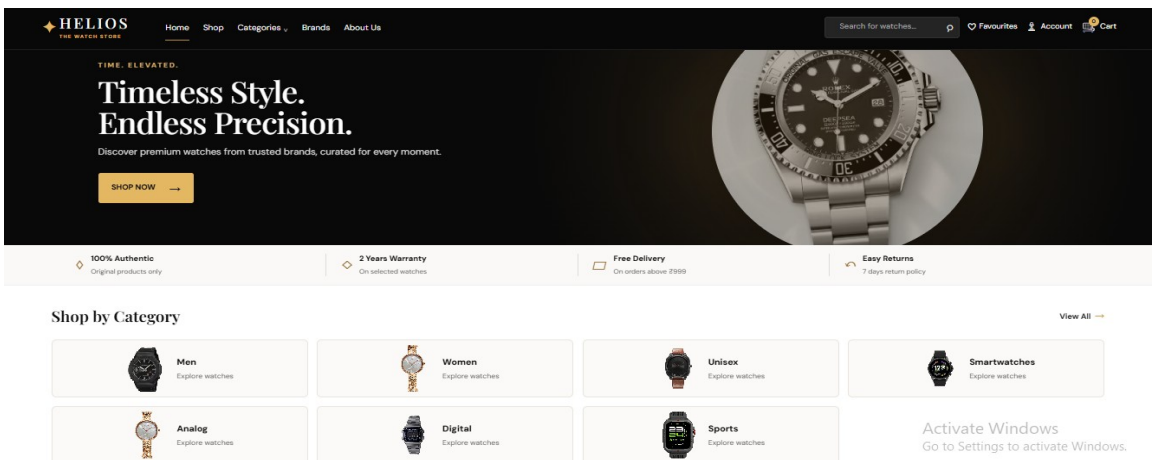
The tests confirmed that the major functionalities of the HELIOS Online Watch Marketplace operate as intended. The application integrates the frontend, Flask backend, and PostgreSQL database to support product browsing, user authentication, favourites, cart management, checkout, and order history.

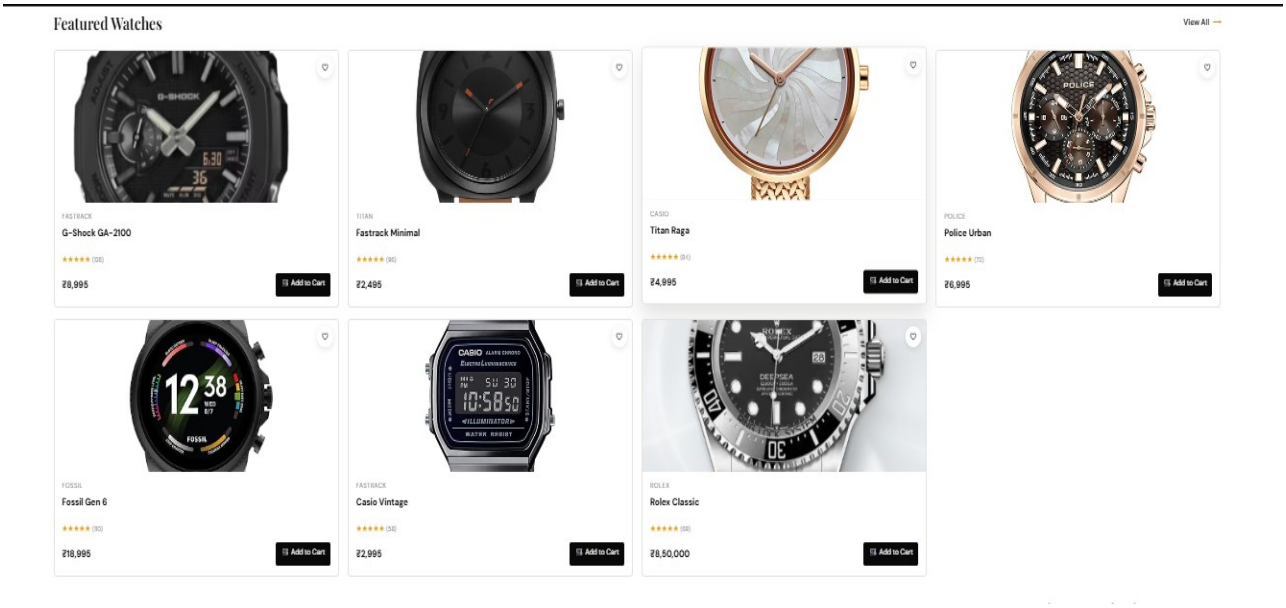
9. Results & Screenshots

The HELIOS Online Watch Marketplace was successfully implemented and deployed as a web-based application. The system allows users to browse watches, view product details, register and log in, manage favourites and shopping carts, and place orders. The application uses a Flask backend connected to a PostgreSQL database.

The following screenshots demonstrate the implemented user interface, major functionalities, and database operations.

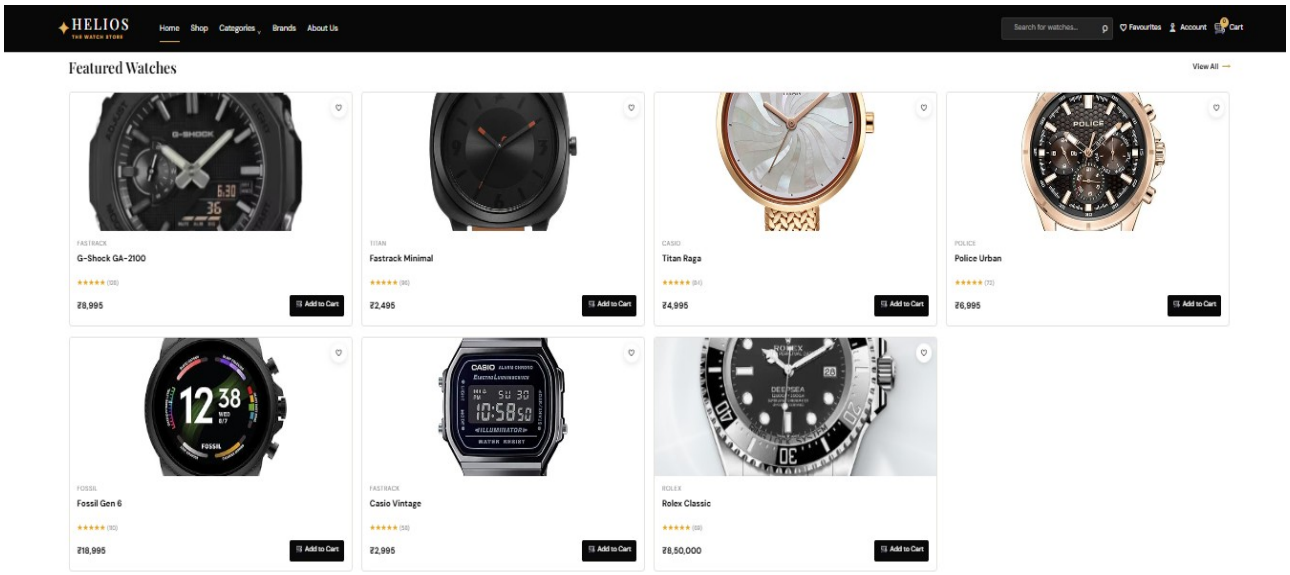
9.1 Homepage





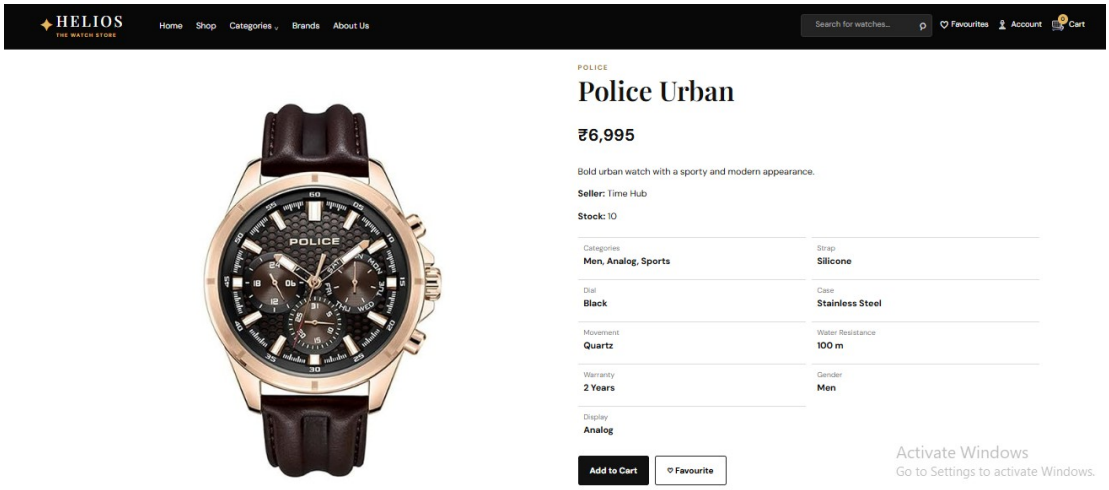
The homepage displays the featured watch, navigation menu, and main sections of the marketplace.

9.2 Product Catalogue



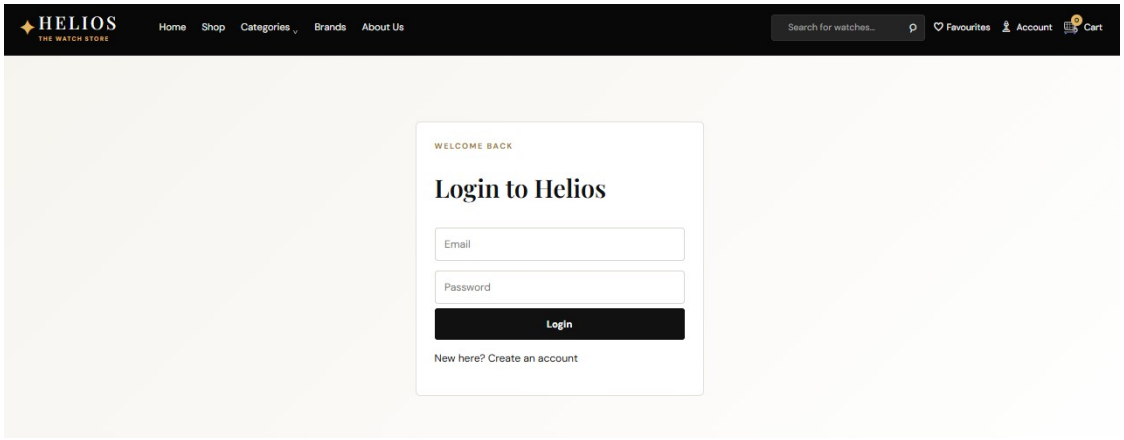
The product catalogue displays the available watches with their images, names, prices, and relevant details.

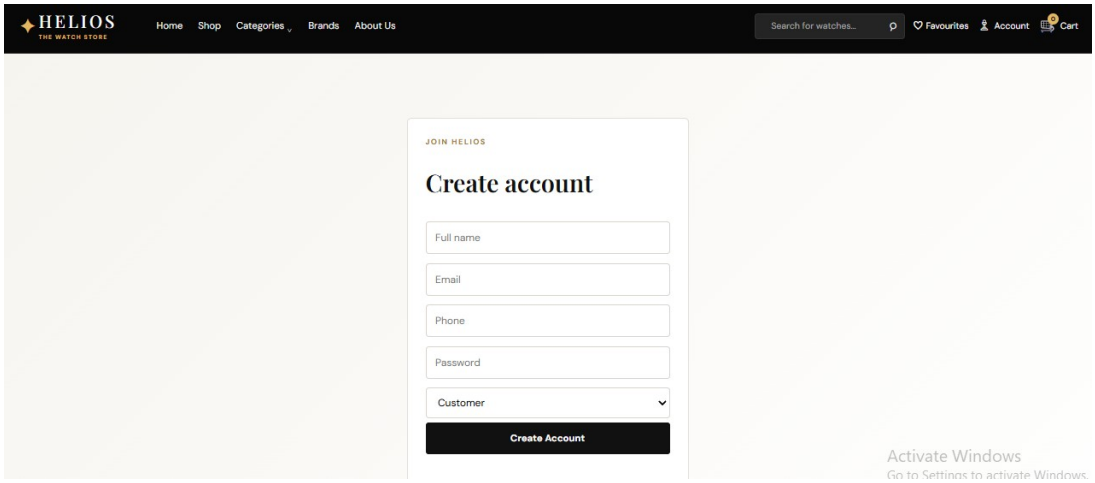
9.3 Product Details



This screen displays the selected watch's price, specifications, and other relevant product information.

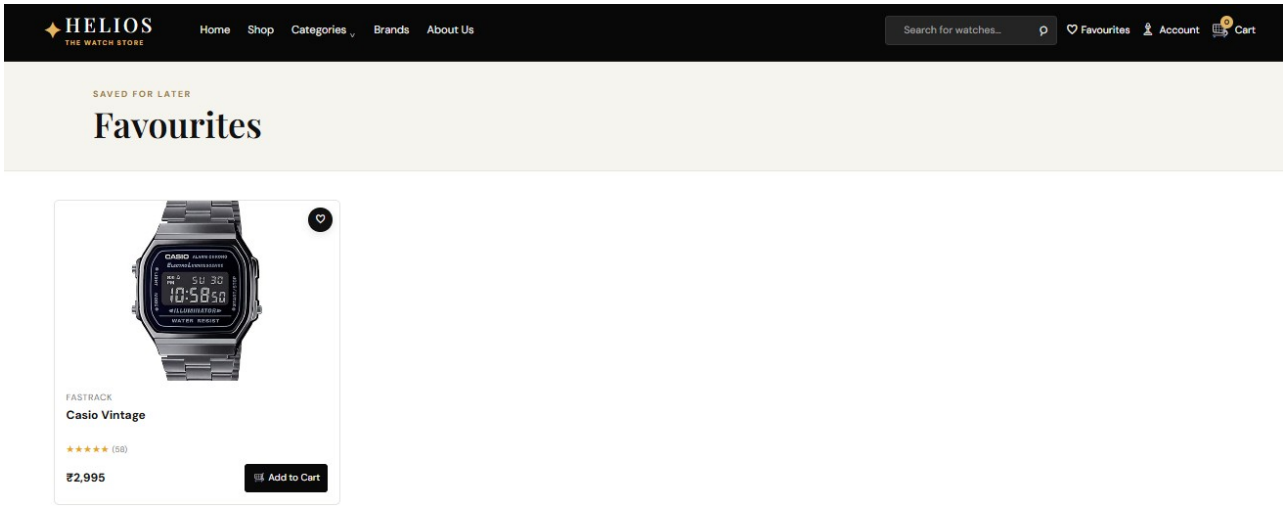
9.4 User Registration and Login

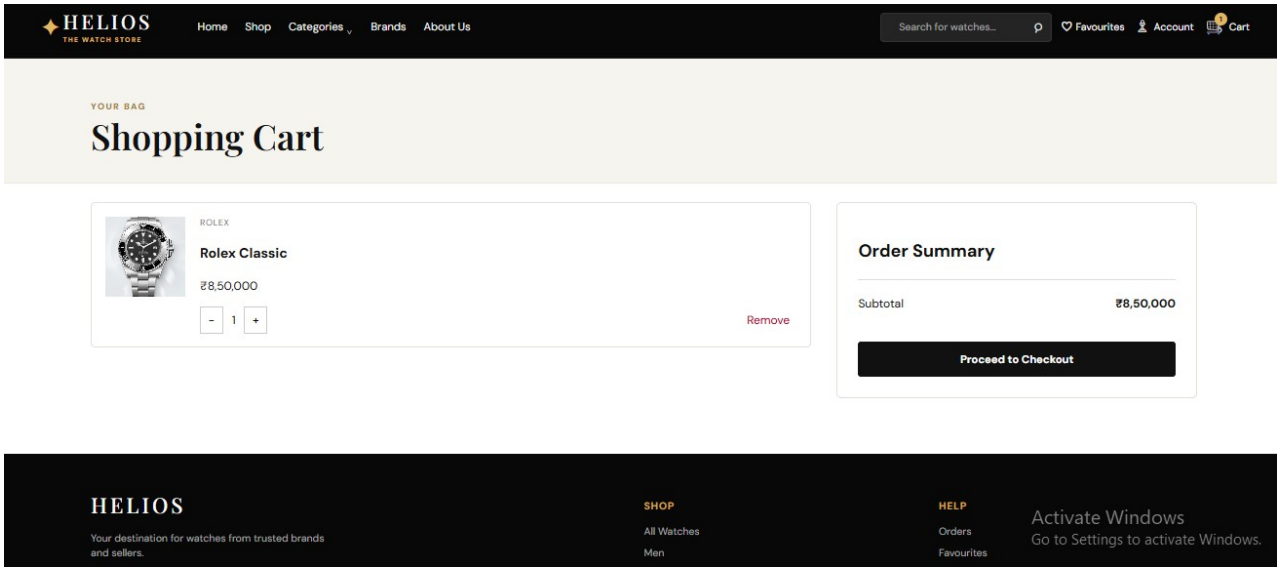




The authentication module allows new users to register and existing users to log in to the system.

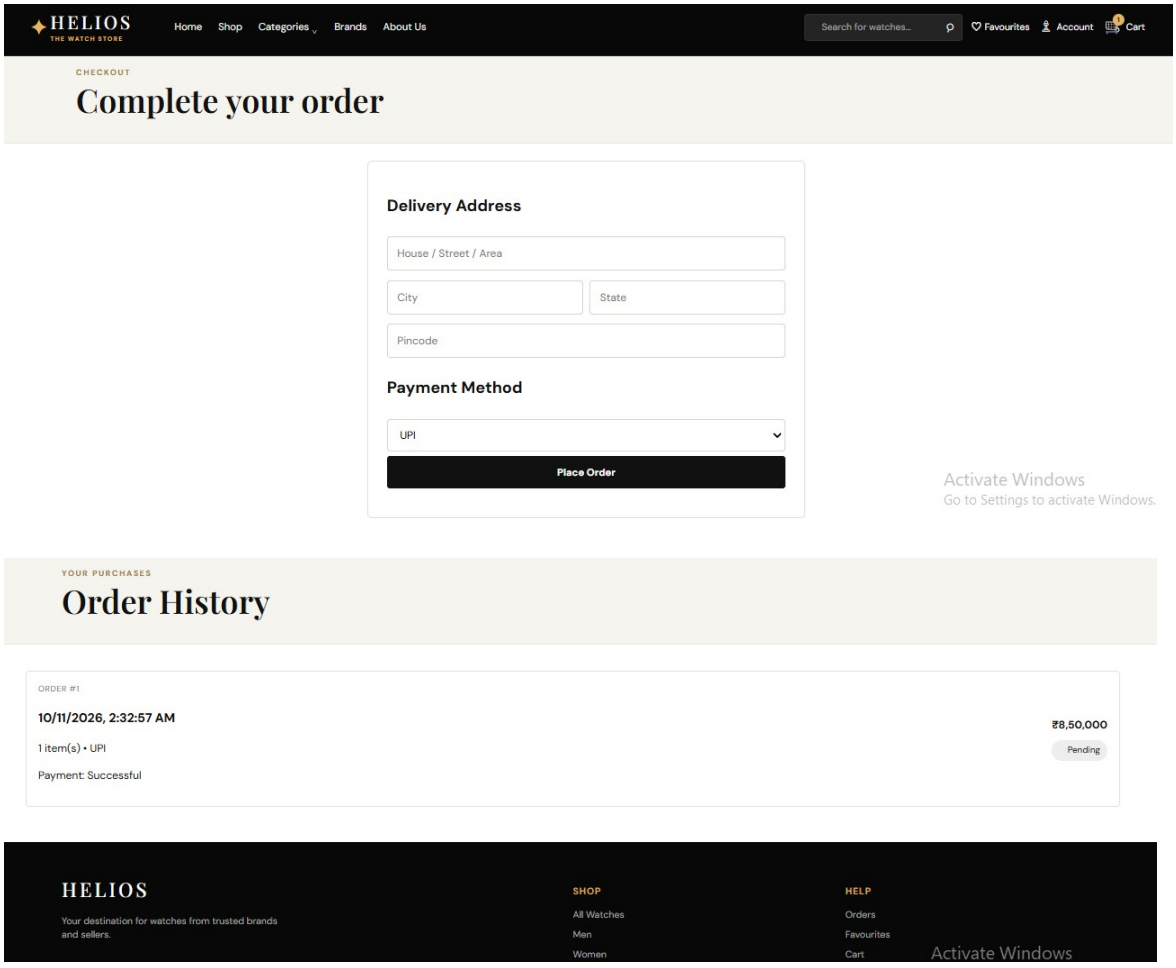
9.5 Favourites and Shopping Cart





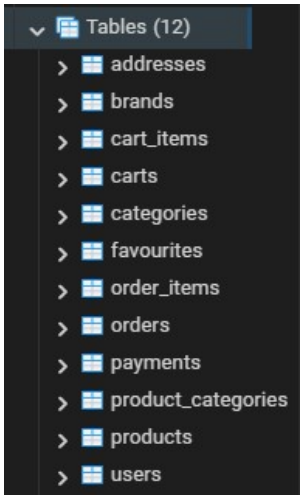
Users can save watches to their favourites and add selected products to their shopping cart.

9.6 Checkout and Order History



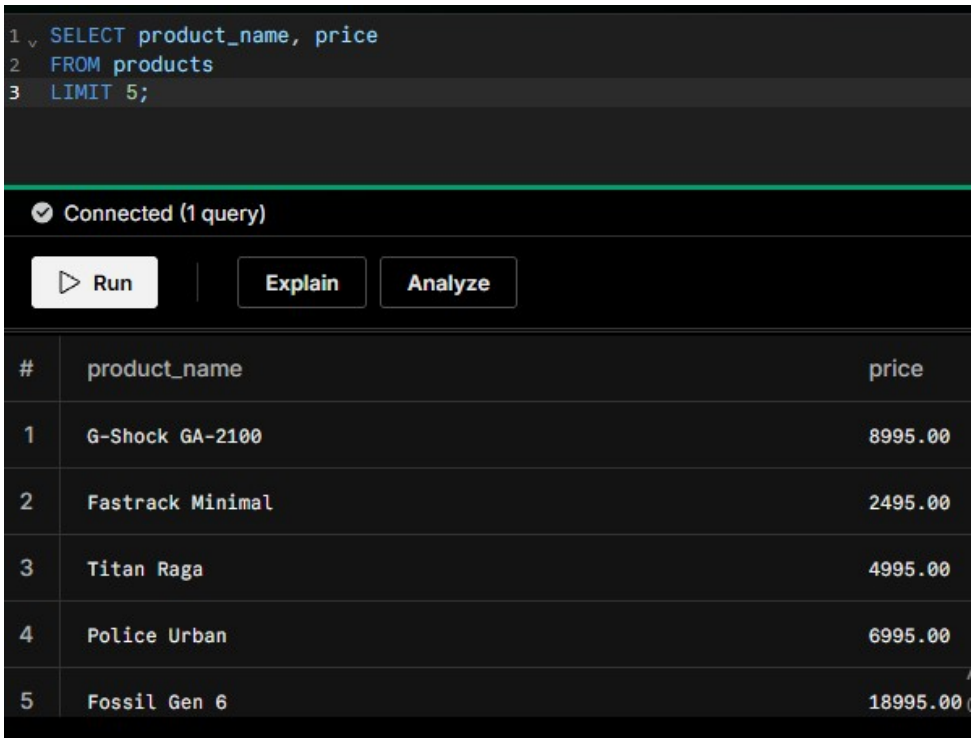
The checkout module collects the required delivery information and payment method. Users can view their placed orders through the order history section.

9.7 Database Tables



The database contains tables for users, products, brands, categories, carts, favourites, orders, and other related information.

9.8 SQL Query Execution



SQL queries are used to retrieve and verify data stored in the PostgreSQL database, demonstrating the database operations performed by the system.

10. Challenges Faced & SolutionsResults & Screenshots

During the development of the HELIOS Online Watch Marketplace, several technical challenges were encountered and resolved.

1. Database Design: Designing relationships between users, products, categories, carts, and orders was challenging. This was addressed by organizing the data into 12 related tables and using primary keys, foreign keys, and intermediate tables.

2. Database Connectivity: Connecting Flask with PostgreSQL required correct database credentials and configuration. This was resolved using the `psycopg` library and environment variables.

3. Frontend and Backend Integration: Retrieving product details and displaying favourites correctly required debugging. The relevant application code was corrected to ensure proper data retrieval and display.

4. Deployment Issues: Initial deployment on Render encountered missing dependencies and database configuration issues. These were resolved by correcting the required dependencies and configuring the hosted PostgreSQL database on Neon.

These challenges helped improve the reliability of the application and provided practical experience in database design, integration, debugging, and deployment.

11. Conclusion & Future Enhancements

11.1 Conclusion

The HELIOS Online Watch Marketplace was successfully developed as a database-driven web application using HTML, CSS, JavaScript, Python Flask, and PostgreSQL. The main objective of providing an organized platform for browsing and purchasing watches was achieved.

The system supports user registration and login, product browsing, search and category filtering, product details, shopping cart management, favourites, checkout, and order history. The database consists of 12 related tables connected through primary keys, foreign keys, and appropriate constraints to maintain data integrity and reduce redundancy. The application was deployed using Render, with PostgreSQL hosted on Neon.

Overall, the project demonstrates the practical application of database management concepts, including relational database design, normalization, constraints, and database connectivity, in developing a functional online marketplace.

11.2 Future Enhancements

The following improvements could be introduced in future versions of HELIOS:

1. **Admin Dashboard:** Add an interface for managing products, stock, users, and orders.
2. **Enhanced Security:** Improve account protection and strengthen input validation.
3. **Online Payment Integration:** Integrate a payment gateway to process real online payments.
4. **Mobile Application:** Develop a mobile application to make shopping more convenient.
5. **Analytics and Reports:** Add sales summaries, popular product analysis, and inventory reports.
6. **Performance Optimization:** Optimize database queries and introduce indexing where necessary to improve performance as the database grows.

These enhancements could make HELIOS more secure, efficient, and suitable for a larger number of users.

References

- Python Documentation: https://docs.python.org/3/
- Flask Documentation: https://flask.palletsprojects.com/
- PostgreSQL Documentation: https://www.postgresql.org/docs/
- MDN Web Docs (HTML, CSS, JavaScript): https://developer.mozilla.org/
- Render Documentation: https://render.com/docs
- HELIOS GitHub Repository: https://github.com/Niharikaah/HELIOS-Watch-Store